# Mask Customer Data in a Test Copy with Oracle Data Safe

**OUT1015 Self-Paced Lab: Activity Guide**

# Table of Contents

**Mask Customer Data in a Test Copy with Oracle Data Safe....1**
Introduction....3
About This Lab....3
What the Setup Creates....3
Objectives....3
Get Started....4
Prerequisites....4
How to Read This Guide....4
Lab 0: Build the Starting State....5
Task 1: Download the Lab Package....5
Task 2: Build the Environment....5
Task 3: Get the Database Passwords....6
Lab 1: Find and Mask the Customer Data....7
Introduction....7
Task 1: See the Problem....7
Task 2: Discover the Sensitive Columns....8
Task 3: Review Who Can Read the Data....8
Task 4: Create the Masking Policy....9
Task 5: Mask the Test Copy....9
Task 6: Prove the Result....10
Checkpoint: What You Proved....11
Repeat the Lab....12
Need Help?....13
Clean Up....14
Acknowledgements....15

Copyright © 2026, Oracle and/or its affiliates.

# Introduction

## About This Lab

At Oracle AI World you watched Asha handle this incident as an instructor demonstration. This guide lets you repeat it in your own OCI tenancy, at your own pace.

Summit Retail's testers need realistic orders to test with. Someone copied the customer records into the test schema, so the testers can now see names, email addresses, and phone numbers. A test account can also still read the source schema. Asha uses Oracle Data Safe to find the sensitive columns and replace their values in the test copy, without breaking the orders. Then she removes the tester's shortcut back to the source.

All records in this lab are invented. No real customer data is used.

**Estimated time:** 75 minutes: 10 minutes of setup, 50 minutes of lab, and 15 minutes of optional clean-up.

## What the Setup Creates

| Resource | Purpose |
|---|---|
| Compartment `aiw-datasafe-lab` | Holds everything in this lab, so clean-up is one command. |
| Autonomous Database `aiw-datasafe-db` | Always Free by default. Holds both schemas. |
| `SUMMIT_SOURCE` schema | 12 synthetic customers and 18 orders, standing in for production. |
| `SUMMIT_TEST` schema | An unmasked copy of the same rows: the problem you fix. |
| `SUMMIT_TESTER` user | Can read the test tables, plus one extra grant on the source customers. |
| Data Safe target `aiw-datasafe-target` | Registers the database with Oracle Data Safe. |

## Objectives

- Discover the sensitive columns in a schema with a Data Safe sensitive data model.
- Find a test account that can read data it should not, with a user assessment and a real query.
- Mask four identity columns in the test copy and confirm that the orders still join.
- Remove the unwanted grant and confirm that the tester can no longer read the source.

Copyright © 2026, Oracle and/or its affiliates.

# Get Started

## Prerequisites

- An OCI tenancy in which you are an administrator, for example a Free Trial or your own paid account.
- Your **home region** selected in the Console. Always Free databases can be created only there.
- A free Always Free Autonomous Database slot. A tenancy can have two. If both are in use, the setup offers a small paid database instead (see the Note in Task 2).
- Oracle Data Safe enabled in the region. If it is not, the setup tells you how to enable it.

## How to Read This Guide

| You see | It means |
|---|---|
| **Bold text** | Text you click or select in the Console. |
| `Code text` | A value you type, or a name you look for. |
| A dark box | Commands to run in Cloud Shell. Lines after the box show the expected output. |
| **Expected result** | What to check before you move on. |

Copyright © 2026, Oracle and/or its affiliates.

# Lab 0: Build the Starting State

One script builds the starting state: the database, the two schemas, the tester, and the Data Safe registration. You run it in OCI Cloud Shell, which signs in as you, so you don't need an API key.

**Estimated time:** 10 minutes.

## Task 1: Download the Lab Package

1. Sign in to the OCI Console, and select your **home region** in the header.
2. In the Console header, click the **Developer tools** icon, and select **Cloud Shell**.
3. Download and extract the package:

```
$ REPO=https://github.com/SaurabhOCI/aiworld-2026-zero-trust-labs
$ ZIP=aiw-datasafe-lab.zip
$ wget -q -O ~/$ZIP $REPO/raw/main/self-paced/$ZIP
$ unzip -o -q ~/$ZIP -d ~/aiw-datasafe-lab && cd ~/aiw-datasafe-lab
$ ls
```

```
README.md  lab.py
```

## Task 2: Build the Environment

1. Run the build. It creates the compartment in your tenancy root:

```
$ python3 lab.py build
```

```
Creating compartment aiw-datasafe-lab
Creating Autonomous Database aiw-datasafe-db (Always Free)
Database: AVAILABLE
Seeded SUMMIT_SOURCE: 12 customers, 18 orders (synthetic)
Registering Data Safe target aiw-datasafe-target
Data Safe target: ACTIVE
...
Data Safe lab: READY
```

Copyright © 2026, Oracle and/or its affiliates.

> **Note**
>
> To create the lab under another compartment, add `--parent <compartment-OCID>`.
>
> If the build reports that no Always Free database is available, run `python3 lab.py build --paid`. It creates a 2-ECPU database that is charged by the hour until you run the clean-up.
>
> If the build stops with a message, fix what it says and run the same command again. It continues where it stopped.

## Task 3: Get the Database Passwords

1. Show the passwords for Database Actions. Keep them private:

```
$ python3 lab.py passwords
```

```
Database Actions sign-in (keep these private):
  ADMIN          Aa9_...
  SUMMIT_TESTER  Aa9_...
```

> **Expected result**
>
> The last line of the build is `Data Safe lab: READY`. You can run `python3 lab.py ready` at any time to check the starting state again.

Copyright © 2026, Oracle and/or its affiliates.

# Lab 1: Find and Mask the Customer Data

## Introduction

The setup put Summit Retail's customers into both schemas. In this lab you work the incident the way Asha did: look at the problem, find the sensitive columns, see who can read them, mask the test copy, and close the extra access.

**Estimated time:** 50 minutes.

## Task 1: See the Problem

1. From the navigation menu, select **Oracle Database** > **Autonomous Database**. Select the compartment `aiw-datasafe-lab`, and click `aiw-datasafe-db`.

2. Click **Database actions**, and then **SQL**. Sign in as `ADMIN` with the password from Lab 0, Task 3.

3. Run this query in the worksheet:

```
$ select customer_id, first_name, last_name,
  email, phone_number, city
$ from SUMMIT_TEST.CUSTOMERS
$ where customer_id <= 3 order by customer_id;
```

```
1  Asha  Rao    asha.rao@example.com    202-555-0101  Seattle
2  Maya  Shah   maya.shah@example.com   202-555-0102  Austin
3  Ravi  Patel  ravi.patel@example.com  202-555-0103  Boston
```

4. Check the order data that the testers need:

```
$ select count(*) as joined_orders,
  sum(o.quantity * o.unit_price) as order_total
$ from SUMMIT_TEST.ORDERS o
$ join SUMMIT_TEST.CUSTOMERS c
  on c.customer_id = o.customer_id;
```

```
18  2200
```

> **Expected result**
>
> The test schema shows the same names, emails, and phone numbers as the source. Renaming the environment to test changed nothing about the data.

Copyright © 2026, Oracle and/or its affiliates.

## Task 2: Discover the Sensitive Columns

1. From the navigation menu, select **Oracle Database** > **Data Safe - Database Security**, and then click **Data discovery** > **Sensitive data models**. Select the compartment `aiw-datasafe-lab`.

2. Click **Create sensitive data model**, and enter these values:

| Field | Value |
|---|---|
| Name | `aiw-sensitive-data` |
| Compartment | `aiw-datasafe-lab` |
| Target database | `aiw-datasafe-target` |
| Schemas | Select only `SUMMIT_TEST` |
| Sensitive types | Keep the default selection |

3. Create the model, and wait until discovery finishes. It takes a few minutes.

4. Open the model, and look at the sensitive columns it found.

> **Expected result**
>
> Discovery finds `FIRST_NAME`, `LAST_NAME`, `EMAIL`, and `PHONE_NUMBER` in `SUMMIT_TEST.CUSTOMERS`. It may also report `CITY`.
>
> Discovery is a starting point, not a complete privacy review. You decide which columns to mask. In this lab you keep `CITY` for regional reports and `CUSTOMER_ID` for the joins.

## Task 3: Review Who Can Read the Data

1. In Data Safe, click **Security center** > **User assessment**, and open the latest assessment for `aiw-datasafe-target`. If no assessment has finished yet, click **Refresh** and wait for it.

2. In the user list, filter for `SUMMIT_TESTER`. The account has no administrative roles, and its risk is low.

3. Now test what the account can actually read. In Database Actions, sign out and sign in again as `SUMMIT_TESTER` with its password from Lab 0, and run:

```
$ select count(*) from SUMMIT_SOURCE.CUSTOMERS;
```

```
12
```

Copyright © 2026, Oracle and/or its affiliates.

> **Expected result**
>
> A tester with no administrative role can still read the source customers. A low-risk rating does not mean the account has only the access it needs.

## Task 4: Create the Masking Policy

1. Return to your sensitive data model `aiw-sensitive-data`, and click **Actions** > **Create masking policy**. Name it `aiw-mask-test-copy`, keep the compartment `aiw-datasafe-lab`, and create it.

2. Wait until the policy is **Active**, open it, and go to **Masking columns**. For each column below, click the three dots, select **View/Edit masking format**, set the format, and apply it:

| Column | Masking format |
|---|---|
| `FIRST_NAME` | Random String, length 8 to 8 |
| `LAST_NAME` | Random String, length 8 to 8 |
| `EMAIL` | Random String, length 10 to 10, then **Add format entry**: Fixed String `@example.com` |
| `PHONE_NUMBER` | Fixed String `202-555-01`, then **Add format entry**: Random Number 50 to 99 |
| `CITY`, if listed | Disable masking for this column |

3. Click **Actions** > **Save Masking Formats**. Then reload the policy, and check that exactly four columns are enabled with the formats above.

> **Note**
>
> Format entries are joined in order, so the email becomes ten random characters followed by `@example.com`, and the phone number keeps its fictional `202-555-01` prefix.

## Task 5: Mask the Test Copy

1. In Data Safe, click **Data masking** > **Pre-masking check**. Select the target `aiw-datasafe-target` and the policy `aiw-mask-test-copy`, leave the optional credentials empty, and run the check. Wait for it to finish, and fix any failed checks before you continue.

Copyright © 2026, Oracle and/or its affiliates.

2. Open the policy `aiw-mask-test-copy`, and click **Mask data**. Select the target `aiw-datasafe-target`, read the confirmation, and submit. Leave the optional seed and credentials empty.

3. Wait until the work request shows **Succeeded**, and open the masking report.

> **Expected result**
>
> The report shows 4 columns and 48 values masked (4 columns for 12 customers).

## Task 6: Prove the Result

1. In Database Actions, as `ADMIN`, run the two queries from Task 1 again.

> **Expected result**
>
> The names are random strings, the emails end in `@example.com`, and the phone numbers start with `202-555-01`. The cities and customer IDs are unchanged.
>
> The order query still returns `18` and `2200`: the test data is still useful.

2. Check that the source is untouched:

```
$ select customer_id, first_name, last_name, email
$ from SUMMIT_SOURCE.CUSTOMERS
$ where customer_id <= 3 order by customer_id;
```

```
1  Asha  Rao    asha.rao@example.com
...
```

3. Remove the tester's extra grant:

```
$ revoke select on SUMMIT_SOURCE.CUSTOMERS
  from SUMMIT_TESTER;
```

4. Sign out, sign in again as `SUMMIT_TESTER`, and run both queries:

```
$ select count(*) from SUMMIT_TEST.ORDERS;
$ select count(*) from SUMMIT_SOURCE.CUSTOMERS;
```

```
18
ORA-00942: table or view does not exist
```

Copyright © 2026, Oracle and/or its affiliates.

> **Expected result**
>
> The tester can still use the masked test data, but can no longer read the source. A login or network error is not the same result; check that you signed in as `SUMMIT_TESTER`.

## Checkpoint: What You Proved

| Question | Before | After |
|---|---|---|
| Can testers see real identities in test? | Yes | No: four columns replaced |
| Do the test orders still join? | 18 orders, 2200 | 18 orders, 2200 |
| Is the source changed? | - | No |
| Can the tester read the source? | Yes (12 rows) | No (ORA-00942) |

- Discovery, masking, and least privilege solve different parts of the problem. Which one would have caught the extra grant?
- Why keep `CUSTOMER_ID` unmasked in this lab?

> **Checkpoint**
>
> **1.** None of the Data Safe features removed the grant. The user assessment and a real query showed it; the `REVOKE` fixed it.
>
> **2.** The orders join on `CUSTOMER_ID`. Masking it would break the relationships the testers need. Real data may need key columns masked consistently instead.

Copyright © 2026, Oracle and/or its affiliates.

## Repeat the Lab

To run the lab again from the start, reset it. The reset deletes your sensitive data model, masking policy, and reports, rebuilds the unmasked test copy, and restores the tester's extra grant:

```
$ cd ~/aiw-datasafe-lab
$ python3 lab.py reset
```

```
...
Data Safe lab: READY
```

> **Note**
>
> Always Free databases stop after 7 days without activity. Start yours again from its Console page; Database Actions can take about 5 minutes to come back.

Copyright © 2026, Oracle and/or its affiliates.

# Need Help?

- `Oracle Data Safe is not enabled in this region`: open **Oracle Database** > **Data Safe - Database Security** once, follow the prompt to enable it, and run the build again.
- `Always Free database not available`: you already have two Always Free databases, or you are not in your home region. Switch to the home region, or run `python3 lab.py build --paid`.
- Database Actions refuses the password: run `python3 lab.py passwords` again, and check that you signed in as the user shown.
- Discovery finds fewer columns: check that you selected only `SUMMIT_TEST`, and that the model finished. Mask the four identity columns even if `CITY` is not listed.
- The masking job fails: open the pre-masking report, fix what it lists, and run the check again before masking.
- Anything else: `python3 lab.py ready` lists each check, so you can see what is different from the starting state.

Copyright © 2026, Oracle and/or its affiliates.

# Clean Up

When you are finished, remove everything the lab created. This deletes the Data Safe target, the database, and the compartment:

```
$ cd ~/aiw-datasafe-lab
$ python3 lab.py teardown --yes
```

```
Deleting Data Safe target
Terminating database
Database: TERMINATED
Compartment deletion submitted (OCI finishes it asynchronously).
```

> **Note**
>
> Data Safe keeps some service records, such as audit profiles, for a while after the target is deleted. They do not incur charges.

Copyright © 2026, Oracle and/or its affiliates.

# Acknowledgements


- **Author:** Saurabh Patil, Principal OCI Instructor, Oracle University
- **Workshop:** OUT1015 Zero Trust Security with OCI, Oracle AI World 2026
- **Last updated:** October 2026


Copyright © 2026, Oracle and/or its affiliates.